

# EFFICIENT TRAINING-FREE CONTEXT SCHEDULING FOR LONG-HORIZON ROBOT VIDEO WORLD MODELS

Anonymous authors

Paper under double-blind review

## ABSTRACT

Autoregressive robot video world models based on discrete visual tokens provide a scalable approach to action-conditioned future prediction, but remain unreliable over long rollout horizons. These models typically retain the entire generated history during inference, yet a growing context does not necessarily yield more accurate predictions. We identify a *memory-role conflict* in Full Context decoding: persistent context, recent visual-action states, and stale generated history with diminishing predictive value all compete for a finite attention budget. As the rollout horizon increases, stale history weakens attention to recent states and action conditioning, leading to structural omissions, motion distortion, and prediction drift. We introduce **ReCAP** (Receding Context with Anchor Preservation), a training-free context-scheduling method for long-horizon robot video world models. **ReCAP** keeps the pretrained tokenizer, world model, and visual decoder fully frozen while periodically reorganizing the active context during inference. It preserves the initial context anchor to maintain scene identity, retains a recent visual-action trajectory for locally coherent generation, and removes redundant stale history from the decoding context. The method requires neither additional modules nor parameter updates, preserves the causal structure of visual-action tokens, and bounds context growth over long rollouts. We evaluate **ReCAP** on RT-1, BridgeV2, LIBERO-90, and CALVIN using world models ranging from 0.5B to 8B parameters. The experiments establish three findings: **(I)** **ReCAP** consistently improves visual fidelity, action-conditioned consistency, and dynamic structure preservation across environments, model scales, and rollout horizons; **(II)** on BridgeV2, it reduces LPIPS by 63.3%, while at  $H = 64$  it reduces the logical KV footprint by 73.3% and p50 latency by 6.51%; and **(III)** attention analysis and controlled history interventions support our diagnosis by showing stronger attention to recent states and reduced interference from stale history. *Stable and efficient long-horizon generation requires the right visual-action context, not more history.* Code, data, models, and qualitative examples are available [here](#).

## 1 INTRODUCTION

Robot video world models predict how visual observations evolve under actions, providing a reusable substrate for planning, policy evaluation, and synthetic experience generation (Finn et al., 2016; Ebert et al., 2018; Ha & Schmidhuber, 2018; Hafner et al., 2023). Recent progress in discrete visual representation learning (van den Oord et al., 2017; Mentzer et al., 2024) and autoregressive sequence modeling (Vaswani et al., 2017; Yan et al., 2021) has made it possible to express observations and actions within a unified causal token sequence. This paradigm scales naturally from game environments (Micheli et al., 2023; Bruce et al., 2024) to interactive and robotic worlds (Wu et al., 2024; Ren et al., 2025; Wu et al., 2025; 2026). Yet robot world models must do more than generate plausible short clips: they must preserve scene identity, robot morphology, object state, and action responsiveness over long horizons. These properties often deteriorate during autoregressive generation, limiting world models for long-horizon reasoning and control.

Most token-based video world models use *full-history decoding*, which we denote as *Full Context*: the initial context, action conditions, and every generated visual state remain active throughout the rollout (Yan et al., 2021; Bruce et al., 2024; Wu et al., 2024). This strategy appears natural because additional history should, in principle, provide more evidence for future prediction. Our experiments

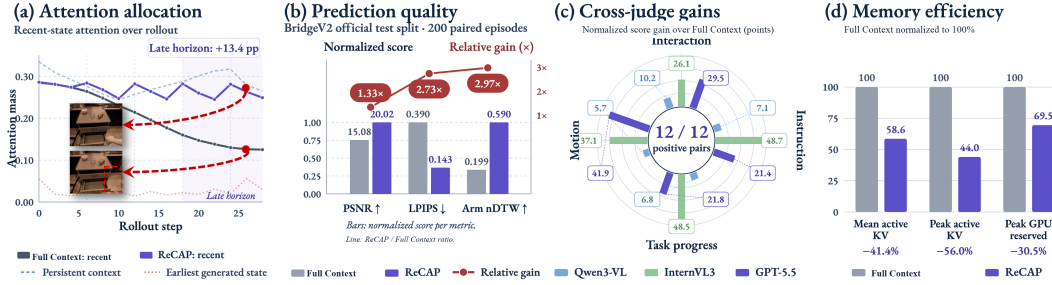


Figure 1: **ReCAP improves long-horizon prediction and memory efficiency.** (a) ReCAP preserves attention to recent visual states. (b) It improves prediction quality on BridgeV2. (c) Three VLM judges favor ReCAP across semantic criteria. (d) It reduces KV-cache and GPU-memory usage. Panels use dedicated samples and extrapolation horizons and are therefore not directly comparable to the fixed-horizon evaluations in Section 5.

reveal the opposite behavior. As shown by the gray curve in Fig. 1(a), the attention assigned to the most recent visual state steadily decreases as the rollout grows, even though distant context and early generated states remain accessible. The resulting predictions are initially plausible but progressively exhibit structural omission, weakened action response, and motion drift. Meanwhile, the active context and KV cache grow monotonically with the rollout horizon. This leads to the central question:

**Key question:** *How should an autoregressive robot video world model organize its inference context to remain dynamically informative and memory efficient over long horizons?*

We interpret this failure through the functional roles of rollout history. The active context contains at least three distinct forms of information: *persistent context* that anchors scene and task identity, *recent visual-action states* that describe the current local dynamics, and *stale generated history* whose incremental predictive value diminishes over time. Persistent context is not itself harmful; removing it can alter viewpoint, layout, or object identity. The problem is that Full Context places all three roles in a single, undifferentiated attention pool. As stale history accumulates, recent states must compete with an ever-growing set of generated tokens for a finite attention budget. We call this phenomenon *memory-role conflict*. The background traces in Fig. 1(a) illustrate that distant information can retain non-negligible attention while the influence of the current state attenuates. This observation complements prior studies of attention sinks and streaming inference in language models (Xiao et al., 2024; Gu et al., 2025a; Barbero et al., 2025), but exposes a distinct requirement in video world models: context management must preserve complete visual-action structure and separate persistent scene information from transient dynamics.

Existing long-context and KV-cache methods primarily identify important individual tokens or compress a language-model cache (Zhang et al., 2023; Liu et al., 2023; Ge et al., 2024; Li et al., 2024). Directly transferring token-level eviction to robot video rollouts is non-trivial because a physical time step is represented by a structured block of visual and action tokens; fragmenting this block can disrupt spatial coherence and causal alignment. Recent long-context video models also recognize that nearby frames and distant context play different roles (Gu et al., 2025b), but typically address this distinction through architectural design or training. We instead ask whether the inference context of an already trained world model can be reorganized without modifying its parameters. This shift turns long-horizon rollout from a problem of retaining *more* history into one of retaining the *right* history.

To this end, we introduce **ReCAP** (**R**eceding **C**ontext with **A**nchor **P**reservation), a training-free context-scheduling method for long-horizon robot video world models. **ReCAP** keeps the pretrained tokenizer, autoregressive world model, and visual decoder fully frozen. At each rollout chunk, it preserves the initial context anchor, retains a recent visual-action trajectory, and removes stale generated history from the active decoding context. Scheduling is performed over complete visual-action blocks, preserving within-frame structure, action alignment, and the causal token grammar seen during pretraining. Compared with language-oriented streaming and token-level cache-eviction methods (Xiao et al., 2024; Zhang et al., 2023; Li et al., 2024), **ReCAP** schedules complete visual-action blocks while retaining global scene identity and the current local trajectory. Unlike architecture- or training-based long-context video models (Gu et al., 2025b), it introduces no additional network, trainable parameter, or post-training stage. The intervention is therefore architecture-preserving and bounds active-context growth independently of the total rollout horizon.

Figure 1 summarizes the resulting gains under its dedicated overview protocols. Context reorganization repeatedly restores the influence of recent visual states, yielding a 13.4-percentage-point advantage over Full Context at the late extrapolation horizon (Fig. 1a). On its 200-pair BridgeV2 evaluation split, ReCAP improves perceptual fidelity, pixel quality, and robot-trajectory consistency, reducing LPIPS by 63.3% (Fig. 1b). Qwen3-VL, InternVL3, and GPT-5.5 consistently prefer ReCAP across interaction quality, instruction following, task progress, and motion naturalness, producing 12/12 positive judge–criterion pairs (Fig. 1c). Under the corresponding efficiency profile, ReCAP reduces mean and peak active KV-cache usage by 41.4% and 56.0%, respectively, and lowers peak reserved GPU memory by 30.5% (Fig. 1d), with near-neutral wall-clock overhead. These overview results use different samples, extrapolation lengths, and aggregation rules from the fixed-horizon analyses in Section 5; their numerical values should therefore not be compared directly. Across RT-1, BridgeV2, LIBERO-90, and CALVIN, the main experiments nevertheless show that the improvements persist across environments and rollout horizons.

Controlled history interventions, attention analysis, and component ablations further support this diagnosis. Holding the current state and recent trajectory fixed while increasing stale history weakens recent-state attention and degrades prediction quality. Removing the context anchor compromises scene identity, whereas discarding recent trajectory information prevents accurate tracking of local dynamics. These results are consistent with memory-role conflict and indicate that stable long-horizon generation requires both persistent scene information and recent dynamic state, without retaining the complete generated past. Our contributions are summarized as follows:

❶ **Empirical diagnosis.** We identify memory-role conflict as a recurring failure mode of Full Context decoding in long-horizon robot video world models, and connect it to recent-state attention decay, structural drift, and unbounded context growth. ❷ **Training-free context scheduling.** We introduce ReCAP, which preserves a persistent context anchor and recent visual–action states while evicting stale generated history, without retraining or architectural modification. ❸ **Multi-level validation.** We demonstrate consistent improvements in prediction quality, semantic consistency, and memory efficiency across robot datasets, supported by controlled interventions, attention analysis, and context-component ablations.

## 2 RELATED WORK

**Robot Video World Models.** World models learn action-conditioned dynamics for planning, policy evaluation, and synthetic experience generation (Ha & Schmidhuber, 2018; Hafner et al., 2023). Early robot video models predict future observations in pixel space (Finn et al., 2016; Ebert et al., 2018), whereas discrete tokenizers and autoregressive Transformers represent observations and actions in a unified causal stream (van den Oord et al., 2017; Yan et al., 2021). Genie, iVideoGPT, and RoboDreamer extend this paradigm to interactive and robotic settings (Bruce et al., 2024; Wu et al., 2024; Ren et al., 2025; Zhou et al., 2024), with reinforcement learning and reward alignment further improving action consistency (Wu et al., 2025; 2026). These studies mainly target representations, architectures, or training, leaving inference-time context organization largely unexplored.

**Long-Horizon Autoregressive Video Modeling.** Autoregressive video world models condition on generated states to support causal, flexible-length rollouts (Yan et al., 2021; Bruce et al., 2024; Wu et al., 2024; Ren et al., 2025). Yet train–inference mismatch compounds errors over time, degrading scene identity, object structure, motion, and action responsiveness (Bengio et al., 2015; Huang et al., 2025; Zhang et al., 2025). Retaining complete history also increases attention and KV-cache costs without guaranteeing better predictions (Gu et al., 2025b; Po et al., 2025). Hierarchical context, context packing, and state-space memory balance local and distant information, but typically require specialized architectures or additional training (Gu et al., 2025b; Zhang et al., 2025). In contrast, ReCAP reorganizes the existing visual–action context of a frozen robot world model at inference time.

**Memory and Context Management.** Memory management spans three main directions. H2O, Scissorhands, FastGen, and SnapKV evict or compress language-model KV caches according to token importance (Zhang et al., 2023; Liu et al., 2023; Ge et al., 2024; Li et al., 2024). Streaming and sliding-window methods preserve initial and recent tokens (Xiao et al., 2024), while attention-sink analyses explain the role of early tokens (Gu et al., 2025a; Barbero et al., 2025). Transformer-XL, Compressive Transformer, and Recurrent Memory Transformer instead propagate or compress hidden states across segments (Dai et al., 2019; Rae et al., 2020; Bulatov et al., 2022).

These approaches mainly operate on language tokens or activations. Robot rollouts instead require complete visual–action blocks, since fragmenting a physical step can disrupt spatial coherence and action alignment. Unlike streaming language models, **ReCAP** preserves a semantic scene anchor together with recent dynamics and schedules complete blocks without architectural modification or additional training.

### 3 PROBLEM FORMULATION

We consider an action-conditioned robot video world model that receives an initial observation sequence  $\mathbf{o}_{1:C}$  and a future action sequence  $\mathbf{a}_{C:C+T-1}$ , and generates future observations  $\hat{\mathbf{o}}_{C+1:C+T}$ . The visual tokenizer  $\mathcal{E}_v$  and action tokenizer  $\mathcal{E}_a$  map each observation and action to  $\mathbf{z}_t = \mathcal{E}_v(\mathbf{o}_t)$  and  $\mathbf{u}_t = \mathcal{E}_a(\mathbf{a}_t)$ , respectively. We concatenate the aligned representations into a visual–action block  $B_t = [\mathbf{z}_t; \mathbf{u}_t]$ . A context policy  $\mathcal{S}$  constructs the active inference context  $\mathcal{C}_t = \mathcal{S}(B_{1:t})$  from the history. The frozen autoregressive world model predicts the next visual representation, and the visual decoder reconstructs the corresponding frame:

$$\hat{\mathbf{z}}_{t+1} \sim p_\theta(\cdot | \mathcal{C}_t), \quad \hat{\mathbf{o}}_{t+1} = \mathcal{D}(\hat{\mathbf{z}}_{t+1}). \quad (1)$$

We seek a context policy that preserves accurate long-horizon prediction under a finite context budget while keeping all model parameters fixed:

$$\mathcal{S}^* = \arg \min_{\mathcal{S}} \mathbb{E} \left[ \sum_{t=C}^{C+T-1} \ell(\hat{\mathbf{o}}_{t+1}, \mathbf{o}_{t+1}) \right], \quad \text{s.t.} \quad |\mathcal{C}_t| \leq M, \quad \theta \text{ fixed.} \quad (2)$$

Here,  $\ell$  measures prediction error and  $M$  denotes the maximum context budget. This objective requires  $\mathcal{S}$  to construct a compact and informative context from the visual–action history, balancing long-horizon prediction quality with inference efficiency.

### 4 METHODOLOGY

**Overview.** Figure 2 illustrates **ReCAP**, a training-free context scheduler for pretrained robot video world models. At step  $t$ , Full Context uses  $P_t^{\text{full}} = A \oplus B_{1:t}$ , whereas **ReCAP** reorganizes this prompt by function: ❶ it preserves the scene anchor  $A$  from the real initial observations; ❷ it retains the most recent  $W$  complete visual–action blocks  $\mathcal{R}_t = B_{\max(1, t-W+1):t}$ ; and ❸ it evicts stale history  $\mathcal{O}_t = B_{1:t-W}$  from the active context and KV cache. It therefore forms  $P_t^{\text{ReCAP}} = A \oplus \mathcal{R}_t$ . The scheduler operates blockwise in token space without changing model parameters or introducing additional decoding and re-encoding.

#### 4.1 RECAP CONTEXT SCHEDULING

Given the real initial observations  $\mathbf{o}_{1:C}$ , we denote their visual tokens as the scene anchor  $A = \mathcal{E}_v(\mathbf{o}_{1:C})$ . Since  $A$  comes from real observations and contains no errors accumulated during autoregressive generation, it provides stable information about robot morphology, object identity, and workspace layout. **ReCAP** preserves  $A$  throughout the rollout and uses it as a persistent scene reference for future prediction.

At generation step  $t$ , the model has produced visual–action blocks  $B_{1:t}$ , where  $B_i = [\mathbf{z}_i; \mathbf{u}_i]$ . **ReCAP** defines the most recent  $W$  blocks as the recent trajectory  $\mathcal{R}_t = B_{\max(1, t-W+1):t}$  and the preceding blocks as stale history  $\mathcal{O}_t = B_{1:t-W}$ ; when  $t \leq W$ ,  $\mathcal{O}_t = \emptyset$ . The recent trajectory describes the current robot state, object changes, and action evolution, whereas the incremental predictive value of stale history decreases as its temporal distance from the current state grows. **ReCAP** therefore removes  $\mathcal{O}_t$  from the active context and constructs

$$P_t^{\text{ReCAP}} = \underbrace{A}_{\text{persistent scene anchor}} \oplus \underbrace{B_{\max(1, t-W+1):t}}_{\mathcal{R}_t: \text{recent trajectory}}, \quad B_i := [\mathbf{z}_i; \mathbf{u}_i]. \quad (3)$$

where  $\oplus$  concatenates token sequences in their original temporal order. This context retains stable global scene information and locally relevant dynamics while excluding generated blocks with indices  $i \leq t - W$ .



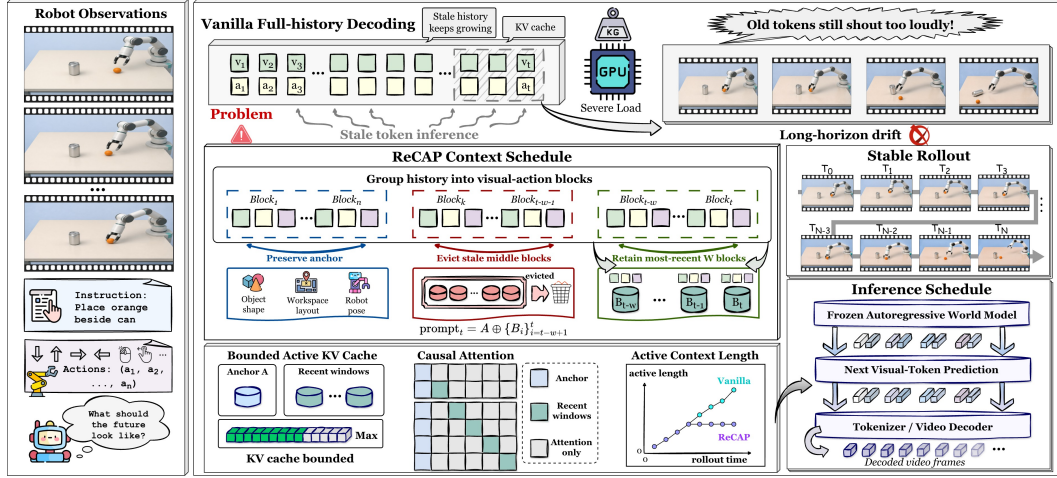


Figure 2: **Overview of ReCAP.** Full Context retains stale generated states, causing attention interference, long-horizon drift, and an ever-growing KV cache. **ReCAP** groups history into complete visual-action blocks, preserves the initial scene anchor and the most recent blocks, and evicts stale middle history. This training-free schedule keeps the active KV cache bounded while maintaining stable autoregressive rollouts.

**ReCAP** updates the recent window online. After the model produces a new block  $B_{t+1}$ , **ReCAP** appends it to the current trajectory and retains only the last  $W$  blocks according to  $\mathcal{R}_{t+1} = \text{Tail}_W(\mathcal{R}_t \oplus B_{t+1})$ . Once the window reaches its maximum size, each new block evicts the oldest block in the window. This process schedules complete visual-action blocks and thus preserves within-frame visual structure, state-action alignment, and the original causal order. **ReCAP** operates directly in token and KV-cache spaces and never decodes generated tokens into images for re-encoding.

## 4.2 BOUNDED-CONTEXT INFERENCE

At generation step  $t$ , **ReCAP** uses  $P_t^{\text{ReCAP}} = A \oplus \mathcal{R}_t$  as the active prompt and predicts the next visual state with the original causal factorization:

$$\hat{z}_{t+1,j} \sim p_\theta(\cdot | A, \mathcal{R}_t, \hat{z}_{t+1,<j}), \quad j = 1, \dots, n_v, \quad (4)$$

where  $n_v$  denotes the number of visual tokens per observation. After predicting all visual tokens, the model combines them with the aligned action tokens to form  $B_{t+1} = [\hat{z}_{t+1}; \mathbf{u}_{t+1}]$ .

**ReCAP** applies the same block-level schedule to the prompt and its KV cache. Let  $\text{KV}(X)$  denote the key-value representations of token sequence  $X$  across all Transformer layers. The active cache at step  $t$  is

$$\text{KV}_t^{\text{ReCAP}} = \text{KV}(A) \oplus \bigoplus_{i=\max(1, t-W+1)}^t \text{KV}(B_i). \quad (5)$$

After generating a new block, **ReCAP** updates the cache as

$$\text{KV}_{t+1}^{\text{ReCAP}} = \text{KV}(A) \oplus \text{Tail}_W \left( \bigoplus_{B_i \in \mathcal{R}_t} \text{KV}(B_i) \oplus \text{KV}(B_{t+1}) \right), \quad (6)$$

where  $\text{Tail}_W$  retains the last  $W$  complete blocks. Thus, the cache reuses the anchor throughout the rollout and replaces the oldest recent block whenever a new block enters a full window.

Let  $n_A$  and  $n_B$  denote the token counts of the anchor and one visual-action block, respectively. Full Context and **ReCAP** use

$$N_t^{\text{full}} = n_A + tn_B, \quad N_t^{\text{ReCAP}} = n_A + \min(t, W)n_B \leq n_A + Wn_B. \quad (7)$$

For a Transformer with  $L$  layers, hidden dimension  $d$ , and  $b$  bytes per cached element, the corresponding KV-cache footprint satisfies

$$M_{\text{KV}}^{\text{ReCAP}}(t) = 2Ldb N_t^{\text{ReCAP}} \leq 2Ldb (n_A + Wn_B). \quad (8)$$

Once  $t \geq W$ , both the active prompt and KV cache remain bounded independently of the total rollout length. **ReCAP** preserves the original causal attention, model parameters, and architecture, and introduces no pixel-space decoding or re-encoding step. Algorithm 1 provides the complete inference procedure in the appendix.

## 5 EXPERIMENTS

**Research questions.** We organize our experiments around four research questions:

**RQ1: Effectiveness and generality.** Does **ReCAP** consistently improve prediction quality across datasets and backbone models?

**RQ2: Long-horizon robustness.** How effectively does **ReCAP** mitigate degradation over increasingly long autoregressive rollouts?

**RQ3: Mechanism and components.** Why does **ReCAP** work, and how do the scene anchor, recent trajectory, and stale-history removal contribute?

**RQ4: Efficiency and scalability.** How does **ReCAP** trade prediction quality against context size, KV-cache memory, and inference cost?

### 5.1 EXPERIMENTAL SETUP

► **Datasets and backbones.** We evaluate **ReCAP** on four robot manipulation datasets: BridgeV2 (Walke et al., 2023), RT-1 (Brohan et al., 2023), LIBERO-90 (Liu et al., 2024), and CALVIN (Mees et al., 2022), which cover diverse scenes, robot embodiments, and manipulation tasks. We use RLVR-World (Wu et al., 2025), iVideoGPT (Wu et al., 2024), and RobotAlign-R1 (Wu et al., 2026) as video world model backbones to examine the generality of **ReCAP** across architectures. We keep all backbone parameters frozen, and **ReCAP** reorganizes only the inference-time context without additional training.

► **Evaluation metrics.** We evaluate generation quality from three complementary perspectives. PSNR, SSIM, and LPIPS measure visual fidelity. Arm Score, Shape IoU, and normalized dynamic time warping (nDTW) evaluate robot shape and trajectory consistency. Qwen3-VL (Bai et al., 2025), InternVL3 (Zhu et al., 2025), and GPT-5.5 (OpenAI, 2026) further assess interaction quality, instruction following, task progress, and motion naturalness.

► **Evaluation protocol and implementation.** For each test episode, we generate paired predictions under identical conditions. We hide method identities and randomize video order during VLM evaluation to reduce evaluation bias. We report averages over all test episodes and compute paired bootstrap 95% confidence intervals. Model execution uses a server with eight NVIDIA H20 GPUs and two 96-core AMD EPYC 9K84 CPUs. World-model serving uses vLLM 0.13.0 in BF16 and eager mode, optimization level `-O0`; CUDA graphs are disabled. Full hardware, software, and evaluation details are provided in the appendix.

### 5.2 OVERALL PERFORMANCE AND GENERALITY (RQ1)

**Quantitative results.** Table 1 shows that **ReCAP** outperforms Full Context in all 36 dataset-backbone-metric combinations. With RLVR-World on BridgeV2, **ReCAP** reduces LPIPS from 0.390 to 0.143 and improves nDTW from 0.199 to 0.590. On RT-1, it increases nDTW and the VLM score by 0.282 and 43.8, respectively. The consistent gains with iVideoGPT and RobotAlign-R1 further demonstrate its generality across models and environments.

**Qualitative results.** Figure 3 qualitatively illustrates the long-horizon improvement. Full Context begins to thin the robot arm at  $t = 6$ , loses it at  $t = 9$ , and freezes by  $t = 15$ , whereas **ReCAP** preserves the robot structure and task progress under the same initial observation, action sequence, and sampling seed. Accordingly, the Arm Score increases from 0.27 to 0.92, nDTW from 0.03 to 0.59, and the blind VLM mean score from 6.00 to 9.25. Additional instruction-conditioned qualitative comparisons across RT-1, CALVIN, and LIBERO-90 are provided in Appendix C.

Table 1: *ReCAP versus Full Context across GPT-style AR robot video world models*. Comparisons pair identical episodes, sampling seeds, and action sequences.  $\Delta$  is the mean paired improvement. RQ1 uses each dataset’s maximum test horizon; subsequent experiments use fixed horizons. Better values are in **bold**.

| Context Strategy     | BridgeV2     |              |             | RT-1         |              |             | LIBERO-90    |              |             | CALVIN       |              |             |
|----------------------|--------------|--------------|-------------|--------------|--------------|-------------|--------------|--------------|-------------|--------------|--------------|-------------|
|                      | LPIPS↓       | nDTW↑        | VLM↑        | LPIPS↓       | nDTW↑        | VLM↑        | LPIPS↓       | nDTW↑        | VLM↑        | LPIPS↓       | nDTW↑        | VLM↑        |
| <i>RLVR-World</i>    |              |              |             |              |              |             |              |              |             |              |              |             |
| Full Context         | 0.390        | 0.199        | 64.8        | 0.251        | 0.276        | 28.1        | 0.269        | 0.551        | 58.9        | 0.287        | 0.528        | 56.4        |
| ReCAP (Ours)         | <b>0.143</b> | <b>0.590</b> | <b>72.6</b> | <b>0.201</b> | <b>0.558</b> | <b>71.9</b> | <b>0.217</b> | <b>0.631</b> | <b>67.8</b> | <b>0.234</b> | <b>0.607</b> | <b>65.1</b> |
| $\Delta$             | -0.247       | +0.391       | +7.8        | -0.050       | +0.282       | +43.8       | -0.052       | +0.080       | +8.9        | -0.053       | +0.079       | +8.7        |
| <i>iVideoGPT</i>     |              |              |             |              |              |             |              |              |             |              |              |             |
| Full Context         | 0.254        | 0.587        | 61.3        | 0.267        | 0.561        | 58.6        | 0.283        | 0.529        | 55.7        | 0.301        | 0.506        | 53.2        |
| ReCAP (Ours)         | <b>0.203</b> | <b>0.665</b> | <b>70.1</b> | <b>0.215</b> | <b>0.639</b> | <b>67.6</b> | <b>0.231</b> | <b>0.609</b> | <b>64.5</b> | <b>0.248</b> | <b>0.586</b> | <b>61.9</b> |
| $\Delta$             | -0.051       | +0.078       | +8.8        | -0.052       | +0.078       | +9.0        | -0.052       | +0.080       | +8.8        | -0.053       | +0.080       | +8.7        |
| <i>RobotAlign-RI</i> |              |              |             |              |              |             |              |              |             |              |              |             |
| Full Context         | 0.226        | 0.634        | 67.2        | 0.239        | 0.608        | 64.5        | 0.255        | 0.576        | 61.8        | 0.272        | 0.554        | 59.1        |
| ReCAP (Ours)         | <b>0.178</b> | <b>0.709</b> | <b>75.8</b> | <b>0.190</b> | <b>0.683</b> | <b>73.1</b> | <b>0.206</b> | <b>0.654</b> | <b>70.2</b> | <b>0.222</b> | <b>0.634</b> | <b>67.4</b> |
| $\Delta$             | -0.048       | +0.075       | +8.6        | -0.049       | +0.075       | +8.6        | -0.049       | +0.078       | +8.4        | -0.050       | +0.080       | +8.3        |

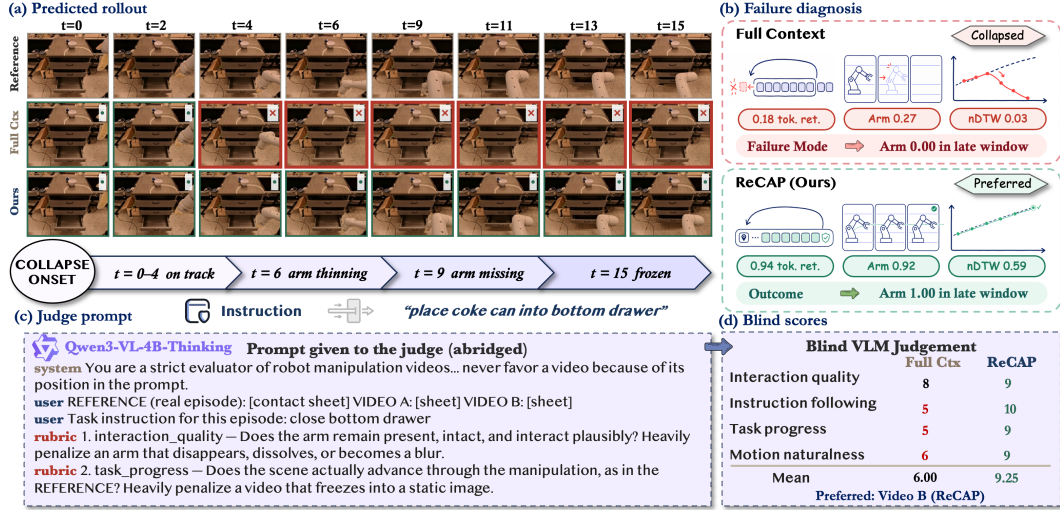


Figure 3: *Qualitative case study*. ReCAP preserves the robot arm throughout the rollout, whereas Full Context collapses; the blind VLM judge prefers ReCAP.

### 5.3 LONG-HORIZON ROBUSTNESS (RQ2)

We evaluate whether ReCAP mitigates error accumulation during long autoregressive rollouts. We generate 64 frames from identical initial observations and ground-truth action sequences, and evaluate prediction quality at  $H = \{8, 16, 32, 48, 64\}$ .

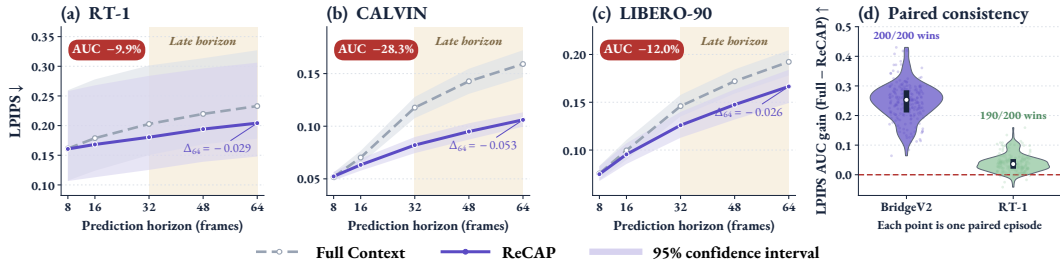


Figure 4: *ReCAP improves long-horizon robustness*. (a–c) LPIPS across prediction horizons on three datasets; shaded bands denote 95% confidence intervals. (d) Paired LPIPS AUC gains on independent BridgeV2 and RT-1 evaluation sets, where positive values favor ReCAP.

**Quantitative results.** Figure 4(a–c) shows a small performance gap at short horizons and an increasingly clear advantage for ReCAP as the rollout grows. ReCAP reduces the LPIPS curve AUC

Table 2: *Long-horizon prediction quality under paired rollouts*. AUC gains are reported with 95% confidence intervals; positive values favor ReCAP. Results at  $H = 64$  are shown as Full Context  $\rightarrow$  ReCAP, and win rates are reported for LPIPS/SSIM.

| Dataset   | LPIPS AUC Gain $\uparrow$ | SSIM AUC Gain $\uparrow$ | $H=64$ LPIPS $\downarrow$        | $H=64$ SSIM $\uparrow$           | Win Rate  |
|-----------|---------------------------|--------------------------|----------------------------------|----------------------------------|-----------|
| RT-1      | +0.0202 [0.0136, 0.0270]  | +0.0207 [0.0131, 0.0286] | 0.233 $\rightarrow$ <b>0.204</b> | 0.703 $\rightarrow$ <b>0.732</b> | 100%/100% |
| CALVIN    | +0.0328 [0.0245, 0.0407]  | +0.0222 [0.0159, 0.0287] | 0.159 $\rightarrow$ <b>0.106</b> | 0.833 $\rightarrow$ <b>0.867</b> | 100%/100% |
| LIBERO-90 | +0.0173 [0.0120, 0.0229]  | +0.0192 [0.0145, 0.0241] | 0.192 $\rightarrow$ <b>0.166</b> | 0.798 $\rightarrow$ <b>0.826</b> | 100%/100% |

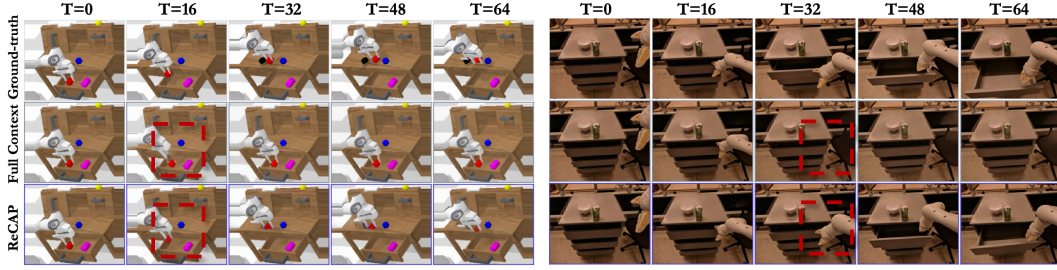


Figure 5: *Qualitative long-horizon comparison*. CALVIN (left) and RT-1 (right) show that ReCAP maintains more consistent scene structure over 64-frame rollouts, while Full Context accumulates visible prediction errors.

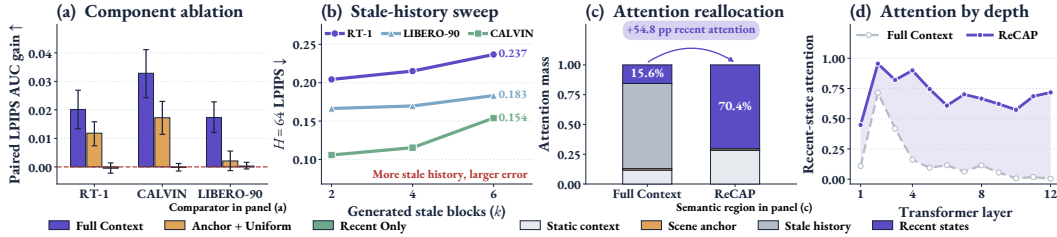


Figure 6: *Mechanism analysis of ReCAP*. (a) Component ablation with 95% bootstrap confidence intervals. (b)  $H=64$  LPIPS after reinserting  $k$  stale blocks. (c) Layer-averaged semantic attention and (d) recent-state attention by layer on CALVIN C06 at  $H=64$ .

by 9.9%, 28.3%, and 12.0% on RT-1, CALVIN, and LIBERO-90, respectively. At  $H = 64$ , it lowers LPIPS from 0.233, 0.159, and 0.192 to 0.204, 0.106, and 0.166, while improving SSIM on all three datasets. Table 2 further shows that every 95% confidence interval excludes zero. Figure 4(d) uses separate BridgeV2 and RT-1 paired evaluation sets rather than the test sets in panels (a–c), and its gains remain concentrated above zero.

**Qualitative results.** Figure 5 compares representative rollouts from CALVIN (left) and RT-1 (right) under identical initial observations and action sequences. Full Context gradually distorts the robot structure and shifts the scene state, whereas ReCAP preserves robot morphology and scene evolution through  $T = 64$ . These examples support the quantitative finding that ReCAP slows long-horizon autoregressive drift.

#### 5.4 MECHANISM AND COMPONENTS (RQ3)

We next ask why bounded context helps. With the context budget fixed, *Anchor + Uniform* keeps the scene anchor but uniformly samples prior blocks, whereas *Recent Only* keeps the recent window without the anchor. We also reinsert stale blocks and inspect attention allocation.

**Component ablation.** Figure 6(a) attributes the gains to stale-history eviction rather than context sampling. Paired LPIPS AUC improves over Full Context by 0.0202, 0.0328, and 0.0173 on RT-1, CALVIN, and LIBERO-90, and over Anchor + Uniform by 0.0118, 0.0173, and 0.0021. Gains over Recent Only are all within  $\pm 0.0005$ , with confidence intervals including zero. Thus, uniformly sampled history cannot replace a contiguous recent window; the anchor preserves the initial scene but provides no measurable LPIPS gain at this horizon.



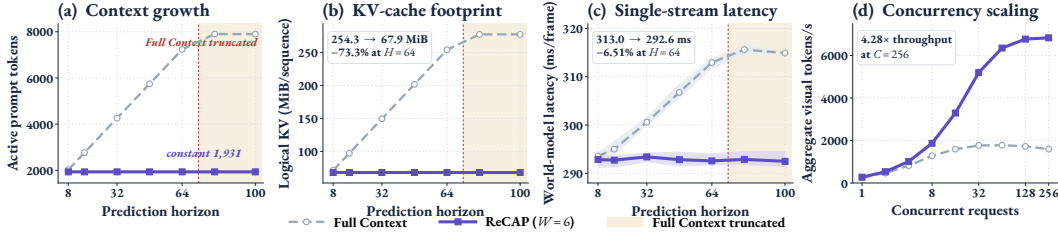


Figure 7: **ReCAP bounds context cost and improves concurrent throughput.** (a) Prompt tokens, (b) logical BF16 KV, (c) p50 world-model latency, and (d) concurrent throughput. Shading marks Full Context truncation; bands in (c) show the range across three datasets.

**Stale-history intervention.** Figure 6(b) causally tests this hypothesis by reinserting  $k$  stale blocks. As  $k$  grows from 2 to 6,  $H=64$  LPIPS rises monotonically from 0.204 to 0.237 on RT-1, 0.106 to 0.154 on CALVIN, and 0.166 to 0.183 on LIBERO-90. Stale history therefore recovers much of the degradation that ReCAP removes.

**Attention analysis.** At  $H=64$ , Full Context assigns 71.5% of attention to stale history and only 15.6% to recent states (Fig. 6c). ReCAP raises recent-state attention to 70.4% (+54.8 percentage points), while anchor attention changes only from 1.37% to 1.45%. This increase appears in all 12 layers and is largest in the deepest layers (Fig. 6d). Together, panels (a–d) show that stale-history eviction redirects attention to the current visual–action state rather than amplifying the anchor.

## 5.5 EFFICIENCY AND SCALABILITY (RQ4)

**Efficiency protocol.** We benchmark one exclusive H20 using a persistent vLLM 0.13.0 BF16 engine in eager/-O0 mode. We set `max_model_len` to 8,192 and reserve 80 visual output tokens per frame. Full Context is capped at 7,900 prompt tokens and is marked as truncated after rollout step 72, whereas ReCAP uses  $W=6$  and remains bounded at 1,931 tokens. Automatic prefix caching is disabled for the main results and evaluated separately as a deployment sensitivity. Each single-stream measurement uses two warm-up runs followed by five measured runs, and prompts are produced by the model’s tokenizer. Latency covers only world-model generation; logical KV excludes allocator overhead, activations, and workspace.

Table 3: **System efficiency summary.**  $H=64$  values are macro averages; throughput uses  $C=256$ .

| Metric               | Full   | ReCAP         | Change |
|----------------------|--------|---------------|--------|
| Tokens ↓             | 7,232  | <b>1,931</b>  | −73.3% |
| Logical KV (MiB) ↓   | 254.25 | <b>67.89</b>  | −73.3% |
| p50 (ms) ↓           | 312.95 | <b>292.57</b> | −6.51% |
| $C=256$ throughput ↑ | 1,596  | <b>6,834</b>  | 4.28×  |

**Bounded cost.** At  $H=64$ , ReCAP reduces the active prompt from 7,232 to 1,931 tokens and logical KV from 254.25 to 67.89 MiB, a 73.3% reduction (Table 3). It also lowers p50 latency from 312.95 to 292.57 ms/frame (−6.51%). Beginning after rollout step 72, Full Context is truncated to at most 7,900 prompt tokens; at  $H=100$ , ReCAP remains bounded at 1,931 tokens (Figure 7(a–c)).

**Concurrent scaling.** The throughput advantage grows from 1.07× at one request to 2.94× at 32 requests and 4.28× at 256 requests (Figure 7(d)). Neither method fails within the tested range; thus, 256 denotes the largest tested concurrency rather than a hard capacity limit.

## 6 CONCLUSION

We presented ReCAP, a training-free context scheduler that preserves scene anchors and recent states while evicting stale history. Across datasets and backbones, ReCAP consistently improves long-horizon prediction. Mechanistic analyses show that it redirects attention from stale history to the current visual–action state rather than amplifying anchor attention. It also reduces logical KV-cache use by 73.3% and achieves 4.28× higher throughput at 256 concurrent requests. Thus, organizing inference-time context can jointly improve prediction quality, efficiency, and scalability without modifying model parameters. This establishes context organization as a key design axis for long-horizon robot world models.



## REFERENCES

- Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, et al. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- Federico Barbero, Álvaro Arroyo, Xiangming Gu, Christos Perivolaropoulos, Michael Bronstein, Petar Veličković, and Razvan Pascanu. Why do LLMs attend to the first token? *arXiv preprint arXiv:2504.02732*, 2025.
- Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems*, pp. 1171–1179, 2015.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Goper, Karol Hausman, Alexander Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2023.
- Jake Bruce, Michael D Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. Genie: Generative interactive environments. In *Forty-first International Conference on Machine Learning*, 2024.
- Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. Recurrent memory transformer. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 2978–2988, Florence, Italy, 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1285.
- Frederik Ebert, Chelsea Finn, Sudeep Dasari, Annie Xie, Alex Lee, and Sergey Levine. Visual foresight: Model-based deep reinforcement learning for vision-based robotic control. In *Robotics: Science and Systems*, 2018.
- Chelsea Finn, Ian Goodfellow, and Sergey Levine. Unsupervised learning for physical interaction through video prediction. *Advances in neural information processing systems*, 29, 2016.
- Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive KV cache compression for LLMs. In *International Conference on Learning Representations (ICLR)*, 2024.
- Xiangming Gu, Tianyu Pang, Chao Du, Qian Liu, Fengzhuo Zhang, Cunxiao Du, Ye Wang, and Min Lin. When attention sink emerges in language models: An empirical view. In *International Conference on Learning Representations (ICLR)*, 2025a.
- Yuchao Gu, Weijia Mao, and Mike Zheng Shou. Long-context autoregressive video modeling with next-frame prediction. *arXiv preprint arXiv:2503.19325*, 2025b.
- David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- Xun Huang, Zhengqi Li, Guande He, Mingyuan Zhou, and Eli Shechtman. Self forcing: Bridging the train-test gap in autoregressive video diffusion. In *Advances in Neural Information Processing Systems*, 2025.
- Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Bo Liu, Yifeng Zhu, Chongkai Gao, Yizhou Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems*, volume 36, 2024.

- Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. In *IEEE Robotics and Automation Letters*, volume 7, pp. 7327–7334, 2022.
- Fabian Mentzer, David Minnen, Eirikur Agustsson, and George Toderici. Finite scalar quantization: VQ-VAE made simple. In *International Conference on Learning Representations*, 2024.
- Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations (ICLR)*, 2023.
- OpenAI. GPT-5.5 system card. <https://openai.com/index/gpt-5-5-system-card/>, April 2026. Accessed: 2026-08-06.
- Ryan Po, Yotam Nitzan, Richard Zhang, Berlin Chen, Tri Dao, Eli Shechtman, Gordon Wetzstein, and Xun Huang. Long-context state-space video world models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 8733–8744, 2025.
- Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.
- Zhongwei Ren, Yunchao Wei, Xun Guo, Yao Zhao, Bingyi Kang, Jiashi Feng, and Xiaojie Jin. VideoWorld: Exploring knowledge learning from unlabeled videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 29029–29039, 2025.
- Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems*, 2017.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- Homer Walke, Kevin Black, Tony Z Zhao, Quan Vuong, Chongyi Zheng, Philippe Hansen-Estruch, Andre Wang He, Vivek Myers, Moo Jin Kim, Max Du, et al. Bridgedata v2: A dataset for robot learning at scale. *arXiv preprint arXiv:2308.12952*, 2023.
- Hao Wu, Yuqi Li, Yuan Gao, Fan Xu, Fan Zhang, Kun Wang, Penghao Zhao, Qiufeng Wang, Yizhou Zhao, Weiyan Wang, et al. Roboalign-r1: Distilled multimodal reward alignment for robot video world models. *arXiv preprint arXiv:2605.03821*, 2026.
- Jialong Wu, Shaofeng Yin, Ningya Feng, Xu He, Dong Li, Jianye Hao, and Mingsheng Long. ivideopt: Interactive videopts are scalable world models. *Advances in Neural Information Processing Systems*, 37:68082–68119, 2024.
- Jialong Wu, Shaofeng Yin, Ningya Feng, and Mingsheng Long. Rlvr-world: Training world models with reinforcement learning. *arXiv preprint arXiv:2505.13934*, 2025.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.
- Wilson Yan, Yunzhi Zhang, Pieter Abbeel, and Aravind Srinivas. Videopt: Video generation using vq-vae and transformers. In *Advances in Neural Information Processing Systems*, 2021.
- Lvmin Zhang, Shengqu Cai, Muyang Li, Gordon Wetzstein, and Maneesh Agrawala. Frame context packing and drift prevention in next-frame-prediction video diffusion models. In *Advances in Neural Information Processing Systems*, 2025.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Siyuan Zhou, Yilun Du, Jiaben Chen, Yandong Li, Dit-Yan Yeung, and Chuang Gan. Robodreamer: Learning compositional world models for robot imagination. *arXiv preprint arXiv:2404.12377*, 2024.

Jinguo Zhu, Weiyun Wang, Zhe Chen, Zhaoyang Liu, Shenglong Ye, Lixin Gu, Hao Tian, Yuchen Duan, Weijie Su, Jie Shao, Zhangwei Gao, Erfei Cui, et al. InternVL3: Exploring advanced training and test-time recipes for open-source multimodal models. *arXiv preprint arXiv:2504.10479*, 2025.

**Algorithm 1** ReCAP inference with anchor preservation

---

**Require:** Scene anchor  $A$ , action tokens  $\mathbf{u}_{1:T}$ , window size  $W$ , frozen world model  $p_\theta$ , and decoder  $\mathcal{D}$

**Ensure:** Generated rollout  $\hat{\mathbf{o}}_{1:T}$

- 1: Initialize the recent trajectory  $\mathcal{R}_0 \leftarrow \emptyset$
- 2: **for**  $t = 0, \dots, T - 1$  **do**
- 3:   Construct the active prompt  $P_t \leftarrow A \oplus \mathcal{R}_t$
- 4:   **for**  $j = 1, \dots, n_v$  **do**
- 5:     Sample  $\hat{z}_{t+1,j} \sim p_\theta(\cdot \mid P_t, \hat{\mathbf{z}}_{t+1,<j})$
- 6:   **end for**
- 7:   Decode  $\hat{\mathbf{o}}_{t+1} \leftarrow \mathcal{D}(\hat{\mathbf{z}}_{t+1})$
- 8:   Form a complete block  $B_{t+1} \leftarrow [\hat{\mathbf{z}}_{t+1}; \mathbf{u}_{t+1}]$
- 9:   Update  $\mathcal{R}_{t+1} \leftarrow \text{Tail}_W(\mathcal{R}_t \oplus B_{t+1})$
- 10: **end for**
- 11: **return**  $\hat{\mathbf{o}}_{1:T}$

---

**A RECAP INFERENCE ALGORITHM**

Algorithm 1 summarizes the block-level inference procedure used by ReCAP.

**B IMPLEMENTATION AND SYSTEMS DETAILS****B.1 HARDWARE ENVIRONMENT**

All model execution is performed on one server with eight NVIDIA H20 GPUs. Each GPU exposes 97,871 MiB (approximately 95.6 GiB) of memory and uses driver 535.161.08. The host contains two AMD EPYC 9K84 96-Core processors, for 192 physical cores and 384 logical CPUs with two threads per core. It runs Linux 5.4.241-1-tlinux4-0017.7 on x86\_64 with glibc 2.28. The RQ4 systems benchmark reserves one H20 exclusively; Ray is used only for job scheduling.

**B.2 SOFTWARE ENVIRONMENT**

Table 4: *Software environment used for model execution and evaluation.* Versions marked “metrics” refer to the separate metric-evaluation environment.

| Component                    | Version                        |
|------------------------------|--------------------------------|
| Python                       | 3.13.11                        |
| PyTorch / torchvision        | 2.9.1 / 0.24.1+d801a34         |
| CUDA (PyTorch build) / cuDNN | 12.9 / 9.1.6.0                 |
| NCCL                         | 2.27.3                         |
| Transformers                 | 4.57.1; 4.57.1.post1 (metrics) |
| vLLM                         | 0.13.0                         |
| Ray                          | 2.51.1 (scheduling only)       |
| Accelerate                   | 1.9.0                          |
| PEFT                         | 0.16.0; 0.17.1 (metrics)       |
| safetensors / flashinfer     | 0.7.0 / 0.5.3                  |
| lpips / piqa                 | 0.1.4 (VGG) / 1.3.2            |
| NumPy / SciPy                | 1.26.4 / 1.17.1                |
| Pillow / imageio             | 12.0.0 / 2.37.0                |
| matplotlib                   | 3.11.1 (rendering only)        |

Figures are rendered locally without model execution on a workstation using Python 3.13.12, matplotlib 3.11.1, Pillow 12.3.0, and NumPy 2.5.1.



### B.3 WORLD-MODEL SERVING AND CONTEXT LIMITS

We serve the world model with vLLM 0.13.0 using `dtype=bfloat16` and one persistent engine. All reported latency and throughput measurements use `enforce_eager=True` at vLLM optimization level `-O0`; CUDA graphs are not enabled. We set `max_model_len=8192` and reserve 80 tokens for the visual output of each generated frame. The Full Context prompt is therefore capped at 7,900 tokens and is explicitly treated as truncated after rollout step 72.

For ReCAP, the scene anchor contains 1,280 tokens, each visual-action block contains 93 tokens, and the recent window uses  $W = 6$ . Its active prompt is consequently bounded by

$$1280 + 93(1 + W) = 1,931 \text{ tokens.} \quad (9)$$

Automatic prefix caching is disabled for the main efficiency results. We evaluate the APC-enabled configuration separately as a deployment-sensitivity experiment.

We report the analytical logical BF16 KV footprint rather than allocated GPU memory. For the evaluated model, the per-token footprint is

$$2 \times 12 \times 12 \times 64 \times 2 = 36,864 \text{ bytes/token,} \quad (10)$$

where the factors correspond to key/value tensors, Transformer layers, KV heads, head dimension, and bytes per BF16 element. This calculation excludes KV-block rounding, allocator metadata, activations, and runtime workspace.

### B.4 CALVIN ADAPTATION AND TRAINING DETAILS

**Dataset and preprocessing.** We use the official CALVIN D-to-D release (`task_D_D`) Mees et al. (2022) and retain its provided training and validation directories without mixing samples across the two splits. The release contains 31 long training rollouts (512,077 raw time steps) and four long validation rollouts (99,022 time steps). Because a single CALVIN rollout can contain tens of thousands of frames, storing each rollout as one compressed archive would make random segment access prohibitively expensive. We therefore partition each rollout into non-overlapping contiguous clips of at most 128 frames, while preserving rollout boundaries. A final remainder shorter than eight frames is discarded. This produces 4,013 training clips with 512,072 frames and 776 validation clips with 99,022 frames. During optimization, the data loader randomly samples a contiguous eight-frame segment from one clip with temporal stride one.

We use the static third-person camera, `rgb_static`, whose native resolution is  $200 \times 200$ . Each frame is isotropically resized to  $256 \times 256$  and padded to  $256 \times 320$  by replicating the left and right image edges (32 pixels per side). This preserves the camera geometry and avoids introducing high-frequency black borders. Pixel values are subsequently scaled to  $[0, 1]$ . We do not apply random crops, color jitter, or other image augmentation.

**Action conversion.** CALVIN provides the normalized seven-dimensional `rel_actions` vector, consisting of Cartesian translation  $(\Delta x, \Delta y, \Delta z)$ , Euler-angle rotation  $(\Delta \phi, \Delta \theta, \Delta \psi)$ , and a binary gripper command. To retain the released RT-1 token geometry, we zero-pad this vector to 13 dimensions rather than modifying the model architecture. Translation is placed in RT-1 slots 4–6, rotation in slots 10–12, and the gripper command in slot 3; slots 0–2 and 7–9 are zero. Thus, each recurrent block remains 93 tokens: 80 dynamics visual tokens followed by 13 action tokens. Each action dimension is independently discretized into 256 uniform bins. The range table is estimated only from the converted training split, with a 2% margin around each observed range; all active normalized dimensions consequently use  $[-1.04, 1.04]$ , while unused zero-filled dimensions use the nominal range  $[-1, 1]$ . Validation actions never contribute to range estimation.

**Stage 1: visual-tokenizer adaptation.** We initialize a `CompressiveVQModelFSQ` from the RT-1 compressive tokenizer released by RLVR-World (Wu et al., 2025). The tokenizer represents the context frame with a  $32 \times 40$  grid (1,280 tokens) and each subsequent frame with an  $8 \times 10$  dynamics grid (80 tokens); both streams use an FSQ vocabulary of 4,375 codes. Training uses eight NVIDIA H20 GPUs, one eight-frame segment per GPU, and no gradient accumulation, giving a global batch size of eight. The first frame is the context frame and the remaining seven frames are dynamics frames.

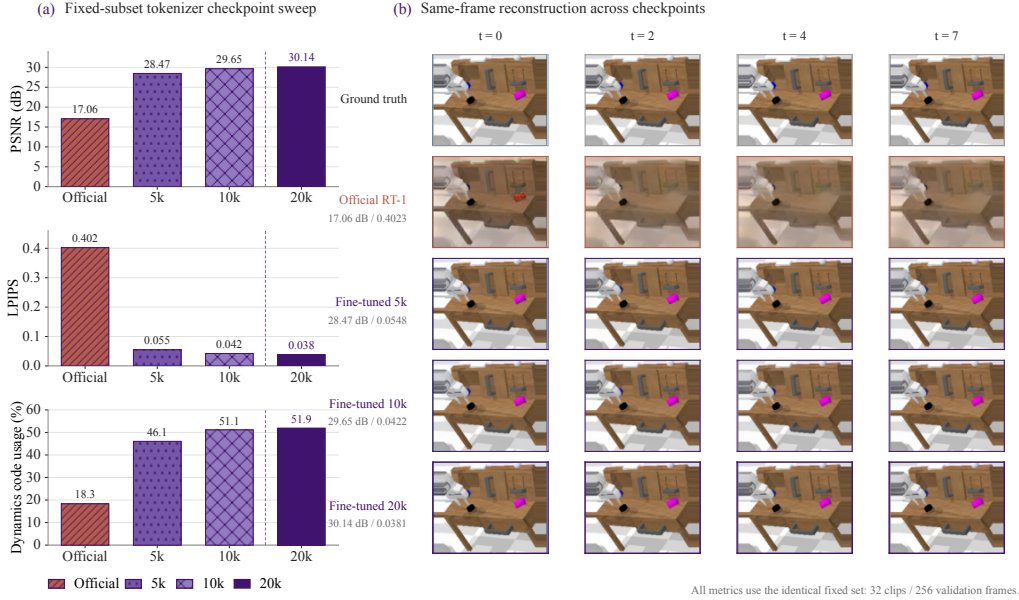


Figure 8: CALVIN tokenizer adaptation. (a) Reconstruction PSNR, LPIPS, and dynamics-code utilization measured on the same fixed set of 32 validation clips (256 frames) for the released RT-1 tokenizer and the 5k, 10k, and 20k checkpoints. Color, hatching, and explicit labels provide redundant encodings. (b) Same-frame reconstructions at  $t \in \{0, 2, 4, 7\}$ . The adapted tokenizer progressively removes the color shift and object disappearance observed with the unadapted RT-1 tokenizer.

We optimize the tokenizer for 20,000 generator updates with AdamW ( $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ ,  $\epsilon = 10^{-8}$ , zero weight decay), a peak learning rate of  $5 \times 10^{-6}$ , cosine decay, 1,000 warmup updates, and gradient clipping at 1.0. Training uses bfloat16 mixed precision and seed 20260903. The objective combines L1 reconstruction, LPIPS perceptual loss, and the context/dynamics FSQ commitment losses. Reconstruction and perceptual terms both have unit weight. We balance context and dynamics contributions in proportion to their temporal occupancy (1/8 and 7/8, respectively). Adversarial training is disabled (discriminator weight zero and discriminator start set beyond the training horizon), so every reported step is an actual tokenizer update. We disable data augmentation, use two data-loader workers per process, log scalar losses every 20 updates, and save checkpoints, validation reconstructions, and image summaries every 1,000 updates.

Checkpoint selection uses a fixed validation subset of 32 clips (eight non-overlapping clips from each of the four validation rollouts), totaling 256 frames. All checkpoints are evaluated on exactly the same frames. The selected 20k tokenizer achieves 30.138 dB PSNR, 0.9457 SSIM, and 0.0381 LPIPS, compared with 17.064 dB, 0.7520, and 0.4023 for the unadapted RT-1 tokenizer. Its dynamics-code utilization is 51.9%, versus 18.3% before adaptation. Training takes approximately 2 h 08 min on eight H20 GPUs. Figure 8 reports the fixed-subset checkpoint sweep together with same-frame reconstructions.

**Stage 2: action-conditioned world-model adaptation.** We freeze the selected 20k CALVIN tokenizer and initialize the autoregressive transformer from the RT-1 multi-step world-model checkpoint released by RLVR-World (Wu et al., 2025). To use the upstream accelerator `load_state` path without changing the trainer, we convert the released Hugging Face checkpoint into an Accelerate checkpoint at step zero. The conversion loads all 111 transformer tensors with strict key and shape matching, initializes fresh AdamW moments, and stores the evaluator, optimizer, scheduler, and RNG state required by the trainer. The transformer has 12 layers, hidden size 768, 12 attention heads, feed-forward size 3,072, vocabulary size 9,008, and a maximum context length of 8,192 tokens.

We train with the `ctx_msp` processor on eight-frame segments. Each training sequence consists of the 1,280-token context representation followed by seven 93-token visual-action blocks. The objective is causal next-token cross-entropy over the tokenized trajectory. We use eight H20 GPUs, per-device batch size one, no gradient accumulation (global batch size eight), AdamW with learning

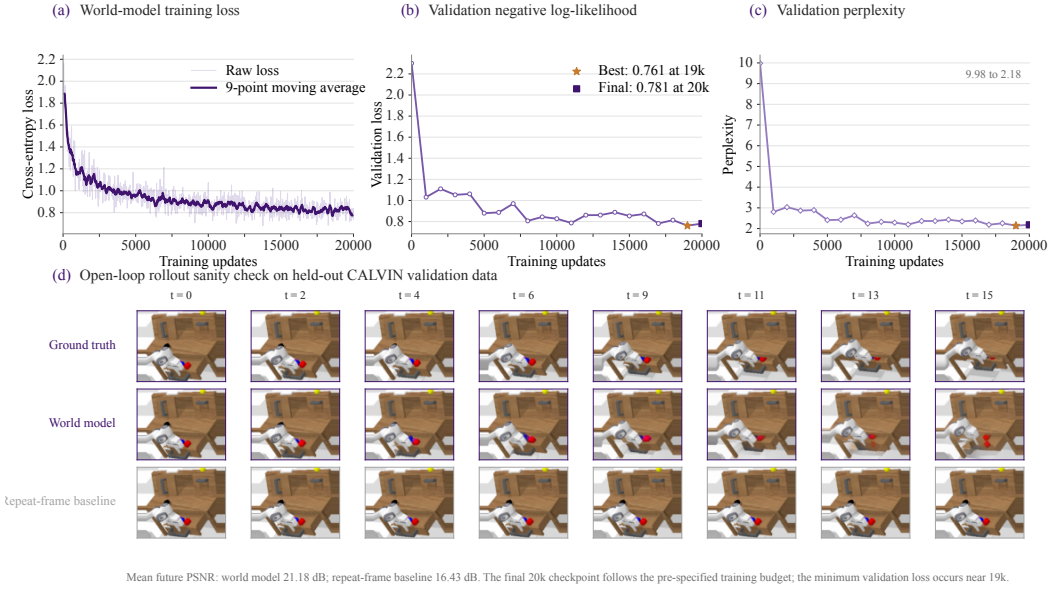


Figure 9: CALVIN world-model adaptation. (a) Raw training loss and its nine-point moving average. (b) Validation negative log-likelihood evaluated every 1,000 updates; the star marks the minimum near 19k and the square marks the pre-specified final 20k checkpoint. (c) Validation perplexity. (d) A 16-frame action-conditioned open-loop rollout on held-out CALVIN validation data. The model receives the first frame and ground-truth actions, then autoregressively feeds back predicted visual tokens. It achieves 21.18 dB mean future PSNR, compared with 16.43 dB for repeating the first frame.

rate  $10^{-5}$  and zero weight decay, a constant schedule with 5,000 warmup updates, gradient clipping at 1.0, bfloat16 mixed precision, and no image augmentation. The tokenizer remains in evaluation mode and is not updated. Training runs for 20,000 updates; checkpoints and validation are produced every 1,000 updates, scalar losses are logged every 20 updates, and online validation is capped at two mini-batches per evaluation to limit overhead.

The final 20k checkpoint is used in all CALVIN rollout experiments. Training takes 33 min 13 s on eight H20 GPUs (approximately 10.2 updates/s). Over training, the monitored validation loss decreases from 2.3010 to 0.7809 (best 0.7611 near 19k), while perplexity decreases from 9.9837 to 2.1835. The 20k checkpoint follows the pre-specified training budget rather than post-hoc selection of the minimum validation value. Figure 9 shows the optimization curves and a held-out open-loop rollout sanity check. Table 5 summarizes the principal hyperparameters.

## B.5 LIBERO-90 ADAPTATION AND TRAINING DETAILS

**Dataset and task-level split.** We use LIBERO-90 Liu et al. (2024), which contains 90 manipulation tasks represented by one HDF5 file per task and 50 demonstrations per task. Because the release does not prescribe a train-validation split for our video-prediction setting, we construct a deterministic task-disjoint split. After lexicographically sorting the 90 task files, we hold out nine evenly spaced tasks and use the remaining 81 tasks for training. This yields 4,050 training episodes with 596,637 time steps and 450 validation episodes with 72,406 time steps. No task appears in both splits. Each HDF5 demo\_<n> group is already one complete demonstration and is therefore retained as one episode rather than subdivided into fixed-length clips. During optimization, the loader samples a contiguous eight-frame segment from an episode with temporal stride one; demonstrations shorter than eight frames would be discarded, although all converted demonstrations satisfy this minimum.

We use the static third-person agentview\_rgb stream and omit the wrist-mounted eye\_in\_hand\_rgb stream. Native  $128 \times 128$  images are isotropically resized to  $256 \times 256$  with bilinear interpolation and padded to  $256 \times 320$  by replicating 32 pixels at each horizontal edge. This matches the RT-1 tokenizer input geometry without stretching the robot or introducing black

Table 5: CALVIN adaptation and training hyperparameters. Batch size denotes the global batch across eight H20 GPUs.

| Setting                          | Tokenizer adaptation      | World-model adaptation      |
|----------------------------------|---------------------------|-----------------------------|
| Initialization                   | RLVR-World RT-1 tokenizer | RLVR-World RT-1 world model |
| Trainable component              | Compressive tokenizer     | Autoregressive transformer  |
| Training updates                 | 20,000                    | 20,000                      |
| Segment / context frames         | 8 / 1                     | 8 / 1                       |
| Global batch size                | 8                         | 8                           |
| Optimizer                        | AdamW                     | AdamW                       |
| Peak learning rate               | $5 \times 10^{-6}$        | $1 \times 10^{-5}$          |
| LR schedule                      | Cosine                    | Constant with warmup        |
| Warmup updates                   | 1,000                     | 5,000                       |
| Weight decay                     | 0                         | 0                           |
| Gradient clipping                | 1.0                       | 1.0                         |
| Precision                        | bfloat16                  | bfloat16                    |
| Image augmentation               | None                      | None                        |
| Checkpoint / validation interval | 1,000                     | 1,000                       |
| GPUs                             | 8 $\times$ NVIDIA H20     | 8 $\times$ NVIDIA H20       |
| Wall-clock time                  | 2 h 08 min                | 33 min 13 s                 |

borders. Pixel values are scaled to  $[0, 1]$  by the training loader. We apply no random crop, color jitter, or other image augmentation.

**Action conversion.** Each LIBERO action is seven-dimensional: Cartesian translation  $(\Delta x, \Delta y, \Delta z)$ , rotational increments  $(\Delta \phi, \Delta \theta, \Delta \psi)$ , and a binary gripper command. We preserve the released RT-1 block structure by zero-padding to 13 dimensions. Translation is assigned to slots 4–6, rotation to slots 10–12, and the gripper command to slot 3; slots 0–2 and 7–9 remain zero. Consequently, every recurrent block remains 93 tokens: 80 dynamics visual tokens followed by 13 action tokens.

Each dimension is independently discretized into 256 uniform bins. Ranges are fitted exclusively on the 596,637 training actions with a 2% margin. The resulting active ranges are  $[-1.04, 1.04]$  for the gripper,  $[-0.975, 0.975]$  for each translation component,  $[-0.35394, 0.38929]$  for roll, and  $[-0.39, 0.39]$  for pitch and yaw; unused slots use  $[-1, 1]$ . The six continuous dimensions occupy 95.7–96.9% of their available bins, while the binary gripper correctly occupies two bins. Validation actions do not affect these ranges.

**Stage 1: visual-tokenizer adaptation.** We initialize `CompressiveVQModelFSQ` from the RT-1 compressive tokenizer released by RLVR-World (Wu et al., 2025). The context frame is represented by a  $32 \times 40$  grid (1,280 tokens), while each subsequent frame uses an  $8 \times 10$  dynamics grid (80 tokens); both streams use an FSQ vocabulary of 4,375 codes. Training uses eight NVIDIA H20 GPUs, one eight-frame segment per GPU, and no gradient accumulation, for a global batch size of eight. The first frame is treated as context and the remaining seven as dynamics frames.

The tokenizer is optimized for 20,000 generator updates with AdamW ( $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ ,  $\epsilon = 10^{-8}$ , zero weight decay), a peak learning rate of  $5 \times 10^{-6}$ , cosine decay, 1,000 warmup updates, and gradient clipping at 1.0. We use bfloat16 mixed precision, seed 20260903, two data-loader workers per process, and no augmentation. The objective combines L1 reconstruction, LPIPS perceptual loss, and context/dynamics FSQ commitment losses. Reconstruction and perceptual terms have unit weight; context and dynamics contributions are balanced by their temporal occupancy (1/8 and 7/8). Adversarial training is disabled by setting its weight to zero and its start step beyond the training horizon. Scalar losses are logged every 20 updates, and checkpoints, validation, and image summaries are produced every 1,000 updates. Training takes 2 h 07 min 55 s on eight H20 GPUs.

For a controlled before/after comparison, we evaluate both the released RT-1 tokenizer and the final 20k tokenizer on exactly the same fixed subset: 32 non-overlapping eight-frame clips from eight held-out episodes, totaling 256 frames. The released tokenizer obtains 25.8858 dB PSNR, 0.9062 SSIM, and 0.2275 LPIPS; after LIBERO adaptation these improve to 31.4548 dB, 0.9669, and 0.0347. Dynamics-code utilization rises from 26.35% to 36.66%. Figure 10 reports these metrics and same-frame reconstructions. The final 20k checkpoint follows the pre-specified training budget;



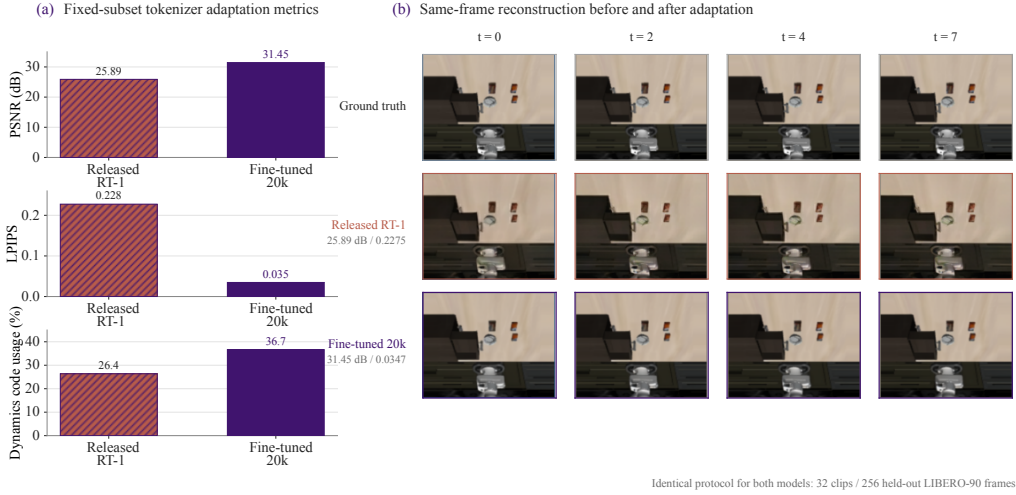


Figure 10: LIBERO-90 tokenizer adaptation. (a) Reconstruction PSNR, LPIPS, and dynamics-code utilization for the released RT-1 tokenizer and the final 20k LIBERO tokenizer, evaluated on the identical fixed set of 32 held-out clips (256 frames). Color, hatching, and explicit labels provide redundant encodings. (b) Same-frame reconstructions at  $t \in \{0, 2, 4, 7\}$ . Adaptation substantially reduces color shift and preserves robot and object structure.

intermediate 12k and 19k images were random validation batches and are not used as a checkpoint sweep.

**Stage 2: action-conditioned world-model adaptation.** We freeze the 20k LIBERO tokenizer and initialize the autoregressive transformer from the RT-1 multi-step world-model checkpoint released by RLVR-World (Wu et al., 2025). The released weights are restored through an exact Accelerate step-zero checkpoint, including model, optimizer, scheduler, and RNG state. The transformer contains 12 layers with hidden size 768, 12 attention heads, feed-forward size 3,072, vocabulary size 9,008, and maximum context length 8,192 tokens.

Training uses the `ctx_msp` processor on eight-frame segments. Each sequence comprises the 1,280-token context frame followed by seven 93-token visual-action blocks. We optimize causal next-token cross-entropy for 20,000 updates on eight H20 GPUs with per-device batch size one, no gradient accumulation, AdamW at  $10^{-5}$  with zero weight decay, a constant schedule with 5,000 warmup updates, gradient clipping at 1.0, bfloat16 mixed precision, and seed 20260903. The tokenizer remains frozen in evaluation mode. We use two data-loader workers per process, log every 20 updates, checkpoint and validate every 1,000 updates, and cap online validation at two mini-batches per evaluation.

World-model training takes 32 min 33 s. Validation loss decreases from 1.7984 to 0.8089 and perplexity from 6.0401 to 2.2453; the minimum validation loss is 0.7482 at approximately 17k updates (perplexity 2.1133). We nevertheless use the pre-specified final 20k checkpoint for all LIBERO experiments. In a held-out high-motion episode, a 16-frame visual open-loop rollout—one observed frame, ground-truth future actions, and autoregressive visual feedback—achieves 19.549 dB mean future PSNR, versus 15.411 dB for repeating the observed frame. At 32 frames the corresponding values are 15.193 and 13.719 dB, illustrating the expected accumulation of open-loop error. Figure 11 shows the optimization curves and 16-frame sanity check. Table 6 summarizes the principal hyperparameters.



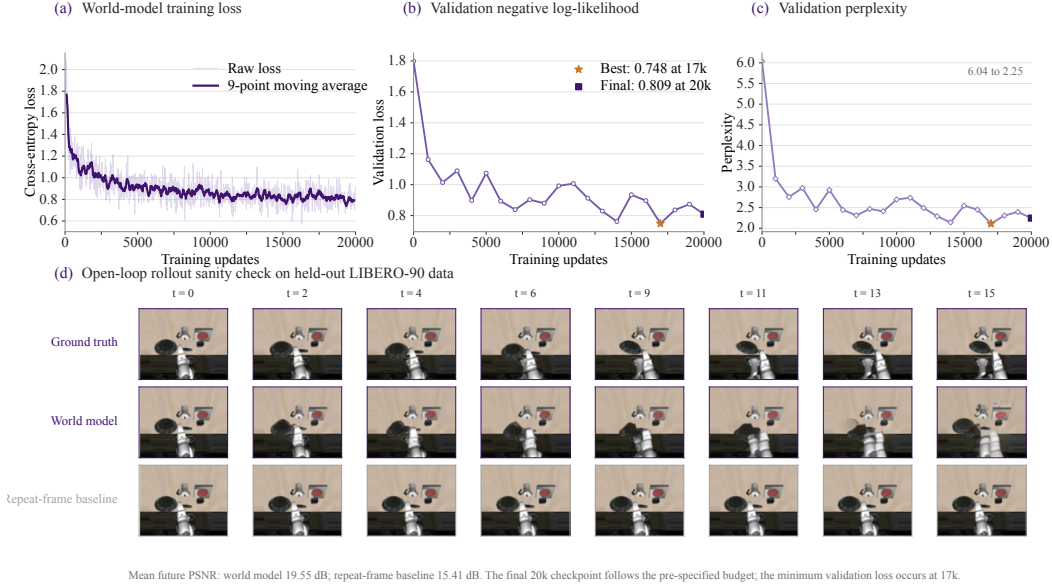


Figure 11: LIBERO-90 world-model adaptation. (a) Raw training loss and its nine-point moving average. (b) Validation negative log-likelihood evaluated every 1,000 updates; the star marks the minimum near 17k and the square marks the pre-specified final 20k checkpoint. (c) Validation perplexity. (d) A 16-frame action-conditioned visual open-loop rollout on a held-out high-motion LIBERO-90 episode. The model observes the first frame, receives ground-truth future actions, and autoregressively feeds back its visual predictions.

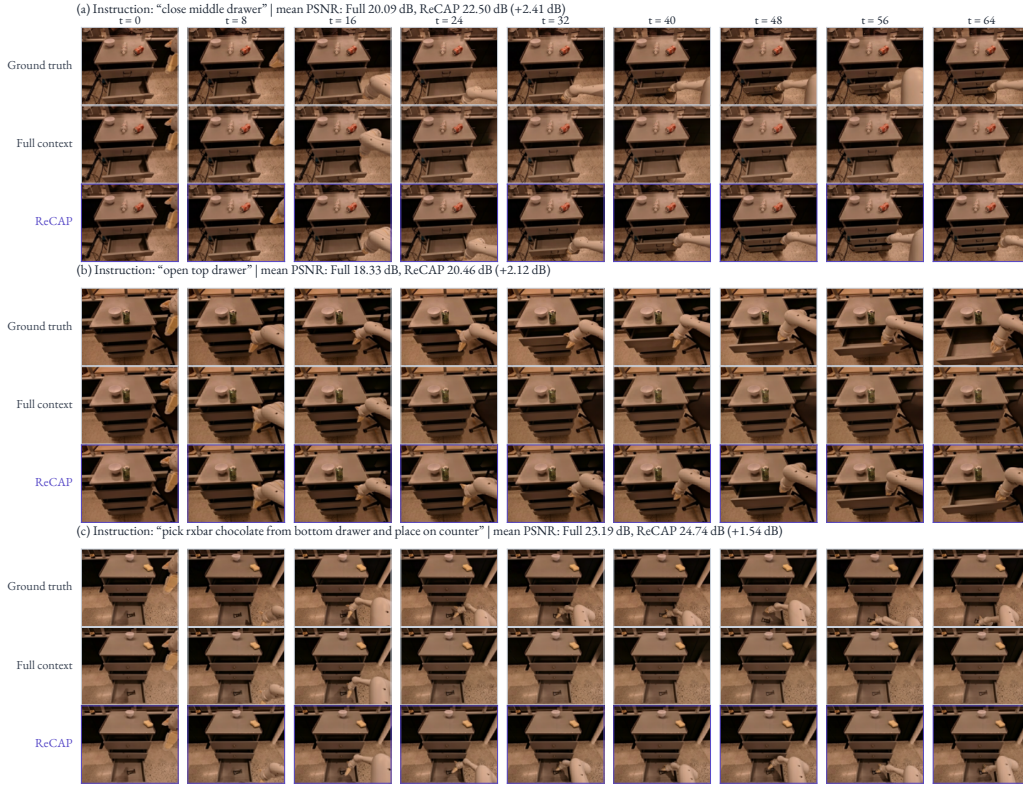
Table 6: LIBERO-90 adaptation and training hyperparameters. Batch size denotes the global batch across eight H20 GPUs.

| Setting                          | Tokenizer adaptation      | World-model adaptation      |
|----------------------------------|---------------------------|-----------------------------|
| Initialization                   | RLVR-World RT-1 tokenizer | RLVR-World RT-1 world model |
| Trainable component              | Compressive tokenizer     | Autoregressive transformer  |
| Training updates                 | 20,000                    | 20,000                      |
| Segment / context frames         | 8 / 1                     | 8 / 1                       |
| Global batch size                | 8                         | 8                           |
| Optimizer                        | AdamW                     | AdamW                       |
| Peak learning rate               | $5 \times 10^{-6}$        | $1 \times 10^{-5}$          |
| LR schedule                      | Cosine                    | Constant with warmup        |
| Warmup updates                   | 1,000                     | 5,000                       |
| Weight decay                     | 0                         | 0                           |
| Gradient clipping                | 1.0                       | 1.0                         |
| Precision                        | bfloat16                  | bfloat16                    |
| Image augmentation               | None                      | None                        |
| Checkpoint / validation interval | 1,000                     | 1,000                       |
| GPUs                             | 8 × NVIDIA H20            | 8 × NVIDIA H20              |
| Wall-clock time                  | 2 h 07 min 55 s           | 32 min 33 s                 |

## C ADDITIONAL QUALITATIVE ROLLOUTS

Figures 12–14 provide additional 64-frame visual open-loop comparisons. Each comparison uses identical initial observations, ground-truth actions, and sampling seed for Full Context and ReCAP. The displayed task instructions are recovered from the original RT-1 episode metadata, CALVIN validation annotations, and LIBERO-90 held-out-task metadata, respectively. Sample identifiers are omitted from the figures. LIBERO-90 frames are vertically flipped for visualization only to undo the OpenGL framebuffer convention; inference arrays and reported metrics are unchanged.

RT-1: strong-gain H=64 evaluation rollouts



All methods use identical initial observations, ground-truth actions, and sampling seed.

Figure 12: **Additional RT-1 qualitative comparisons.** Nine time points are shown for each rollout. Across these three instruction-conditioned examples, ReCAP improves mean PSNR over Full Context by +2.41, +2.12, and +1.54 dB, respectively.

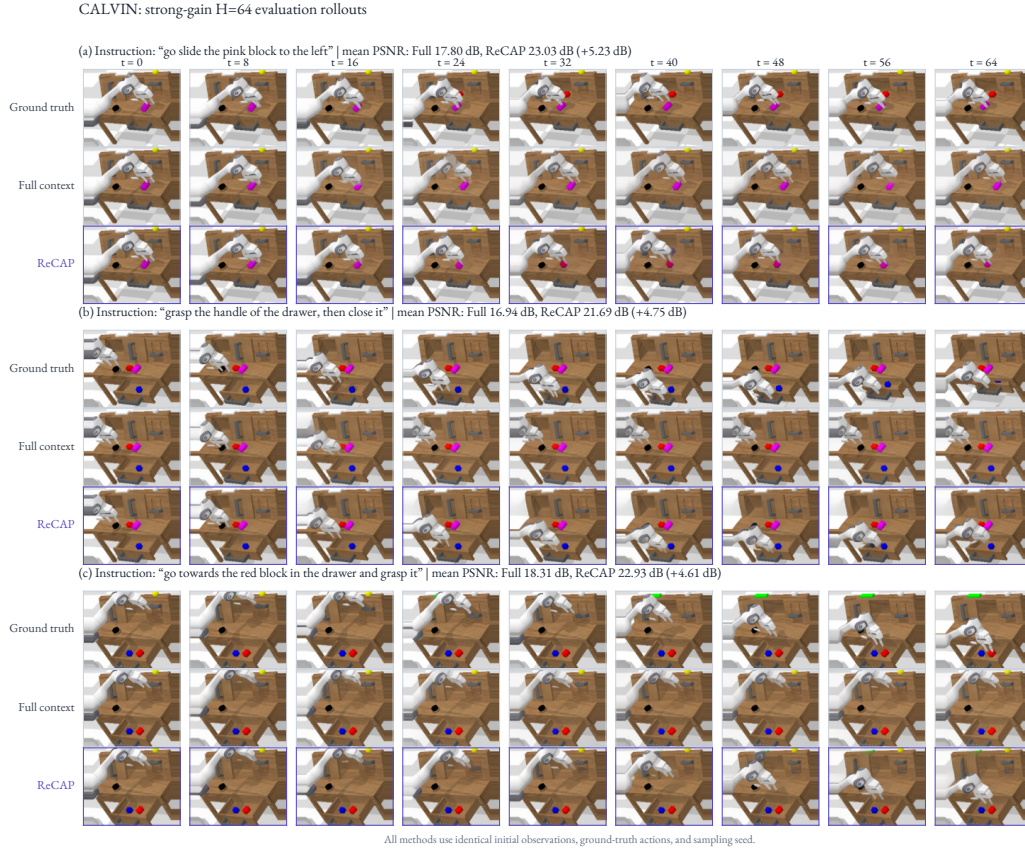


Figure 13: **Additional CALVIN validation comparisons.** The examples are drawn from annotated validation trajectories. ReCAP improves mean PSNR over Full Context by +5.23, +4.75, and +4.61 dB, respectively.

LIBERO-90: strong-gain H=64 evaluation rollouts



Figure 14: **Additional LIBERO-90 held-out-task comparisons.** The three examples use distinct language instructions from the task-disjoint validation split. ReCAP improves mean PSNR over Full Context by +6.04, +3.92, and +2.61 dB, respectively. Frames are vertically flipped for canonical display orientation only.